Contents

1 transform.cpp **1**

1.1 Transform::AddChild(Transform* tr) 1
1.2 Transform::AddComponent(Component* component) 1
1.3 Transform::FindChildByName(std::string name) 2
1.4 Transform::FindComponentByUID(uint64$_t$ id) 2
1.5 Transform::FindTransformByUID(uint64$_t$ id) 2
1.6 Transform::GetChildren() 2
1.7 Transform::GetComponent(std::string name) 2
1.8 Transform::GetGlobalPosition() 3
1.9 Transform::GetGlobalRotation() 3
1.10 Transform::GetGlobalScale() 3
1.11 Transform::GetLocalPosition() 3
1.12 Transform::GetLocalRotation() 3
1.13 Transform::GetLocalScale() 3
1.14 Transform::Rotate(glm::vec3 axis, float amount) 4
1.15 Transform::Scale(glm::vec3 amount) 4
1.16 Transform::Translate(glm::vec3 offset) 4
1.17 Transform::Forward() 4
1.18 Transform::Backward() 5
1.19 Transform::Left() 5
1.20 Transform::Right() 5
1.21 Transform::Up() 5
1.22 Transform::Down() 5

1 transform.cpp

1.1 Transform::AddChild(Transform* tr)

1.1.1 Equivalence Classes

lXX

Eq. Class ID	Param	Description	Representative
Transform::AddChild::EC1.1	tr	a valid pointer	new Transform()
Transform::AddChild::EC1.a	tr	an invalid pointer	nullptr

1.2 Transform::AddComponent(Component* component)

1.2.1 Equivalence Classes

lXX

Eq. Class ID	Param	Description	Representative
Transform::AddComponent::EC1.1	component	a valid pointer	new Component()
Transform::AddComponent::EC1.a	component	an invalid pointer	nullptr

1.3 Transform::FindChildByName(std::string name)

1.3.1 Equivalence Classes

|XX

Eq. Class ID	Param	Description	Representative
Transform::FindChildByName::EC1.1	name	A string	"string"
Transform::FindChildByName::EC1.?	name	The empty string	""
Transform::FindChildByName::EC1.a	name	Not a string	1

- The specification is unclear on whether the empty string is a valid name for a Transform

1.4 Transform::FindComponentByUID(uint64$_t$ id)

1.4.1 Equivalence Classes

|XX

Eq. Class ID	Param	Description	Representative
Transform::FindComponentByUID::EC1.1	id	A positive integer	1
Transform::FindComponentByUID::EC1.2	id	Zero	0
Transform::FindComponentByUID::EC1.a	id	A negative integer	-1

1.5 Transform::FindTransformByUID(uint64$_t$ id)

1.5.1 Equivalence Classes

Equivalence classes same as Transform::FindComponentByUID.

1.6 Transform::GetChildren()

1.6.1 Equivalence Classes

Returns an instance variable that will always be valid.

1.7 Transform::GetComponent(std::string name)

1.7.1 Equivalence Classes

Possible issues with specification:

- The method to get a transform by name is called ‘FindChildByName‘, this method should be updated to match that scheme

  The equivalence classes are the same as Transform::FindChildByName.

1.8 Transform::GetGlobalPosition()

1.8.1 Equivalence Classes

Eq. Class ID	Param	Description
Transform::GetGlobalPosition::EC1.1	parent	parent = nullptr
Transform::GetGlobalPosition::EC1.2	parent	parent is a valid pointer to a Transform
Transform::GetGlobalPosition::EC1.a	parent	parent is a valid pointer to something other than a T

1.9 Transform::GetGlobalRotation()

1.9.1 Equivalence Classes

Equivalence classes same as Transform::GetGlobalPosition.

1.10 Transform::GetGlobalScale()

1.10.1 Equivalence Classes

Equivalence classes same as Transform::GetGlobalScale.

1.11 Transform::GetLocalPosition()

1.11.1 Equivalence Classes

This is a simple getter and only exposes a field.

1.12 Transform::GetLocalRotation()

1.12.1 Equivalence Classes

This is a simple getter and only exposes a field.

1.13 Transform::GetLocalScale()

1.13.1 Equivalence Classes

This is a simple getter and only exposes a field.

1.14 Transform::Rotate(glm::vec3 axis, float amount)

1.14.1 Equivalence Classes

|XX

Eq. Class ID	Param	Description	Representative
Transform::Rotate::EC1.1	axis	axis is a vector with nonzero length	{x=1.0,y=0.0,z=0.0}
Transform::Rotate::EC1.a	axis	axis is a vector with zero length	{x=0,y=0,z=0}
Transform::Rotate::EC1.b	axis	axis is not a vector	1
Transform::Rotate::EC2.1	amount	amount is a float	90.0f
Transform::Rotate::EC2.2	amount	amount is an integer	90
Transform::Rotate::EC2.a	amount	amount is not a number	"90"

1.15 Transform::Scale(glm::vec3 amount)

1.15.1 Equivalence Classes

|XX

Eq. Class ID	Param	Description	Representative
Transform::Scale::EC1.1	amount	Any vector	{x=2.0,y=3.0,z=2.0}
Transform::Scale::EC1.2	amount	The identity element	{x=1.0,y=1.0,z=1.0}
Transform::Scale::EC1.3	amount	The zero vector	{x=0.0,y=0.0,z=0.0}
Transform::Scale::EC1.a	amount	Not a vector	1

1.16 Transform::Translate(glm::vec3 offset)

1.16.1 Equivalence Classes

|XX

Eq. Class ID	Param	Description	Representative
Transform::Translate::EC1.1	amount	Any vector	{x=2.0,y=3.0,z=2.0}
Transform::Translate::EC1.2	amount	The identity	{x=0.0,y=0.0,z=0.0}
Transform::Translate::EC1.a	amount	Not a vector	1

1.17 Transform::Forward()

1.17.1 Equivalence Classes

|XX

Eq. Class ID	Param	Description	Representative
Transform::Forward::EC1.1	rotation (implicit)	Any unit quaternion $x \in \mathbb{H} : \|x\| = 1$	{x=0,y=0,z=0,w=1}
Transform::Forward::EC1.a	rotation (implicit)	A non-unit quaternion $x \in \mathbb{H} : \|x\| \neq 1$	{x=0,y=0,z=0,w=2}

1.18 Transform::Backward()

1.18.1 Equivalence Classes

Equivalence classes same as Transform::Forward.

1.19 Transform::Left()

1.19.1 Equivalence Classes

Equivalence classes same as Transform::Forward.

1.20 Transform::Right()

1.20.1 Equivalence Classes

Equivalence classes same as Transform::Forward.

1.21 Transform::Up()

1.21.1 Equivalence Classes

Equivalence classes same as Transform::Forward.

1.22 Transform::Down()

1.22.1 Equivalence Classes

Equivalence classes same as Transform::Forward.